

Informe: “Presentación 1”

Asignatura: INF331 - Pruebas de software

Profesor: Oscar Rodrigo Reyes Hevia

Grupos: 8 y 7

Repositorio: <https://github.com/validacion-y-verificacion/presentacion>

Link video: <https://youtu.be/XwZORsa4Lsk>

Integrantes:

Javier Norambuena

Bastián Cornejo

Benjamin Cayo

Jest, una herramienta de testing

Que es Jest

Jest es una herramienta de pruebas de software para ambientes de desarrollo que utilizan JavaScript. Como ya se dio a entender, esta es utilizada para el desarrollo de pruebas unitarias y de integración. Esta herramienta se vende como “rápida y segura”, “fácil mocking”, “buen manejo de excepciones” y “buena cobertura de código”.

Alcances de Jest

Jest es una herramienta versátil que permite el desarrollo y ejecución de pruebas unitarias, de integración y de snapshot, las cuales pueden ser realizadas de forma asincrónica.

Otra de las ventajas que posee Jest es la capacidad de simular los comportamientos de objetos y métodos (mockup). Finalmente, Jest es compatible con frameworks de desarrollo escritos en JavaScript, tales como React, Vue, Angular y Node.js.

Proyecto a desarrollar

Detalles comunes

El problema elegido fue el de la biblioteca de Salamanca, el contexto entregado fue: “**La Biblioteca Central de la Universidad de Salamanca** cuenta con un sistema de gestión de biblioteca antiguo, “legado” que ya no tiene soporte y es difícil de mantener. El sistema actual no permite llevar una trazabilidad adecuada de los materiales de estudio (conocer lugar y quién tiene cierto material), y los usuarios (estudiantes y docentes) no siempre encuentran ejemplares a través de las búsquedas en línea, y en el peor de los casos deben realizar las búsquedas y reservas de manera presencial o a través de formularios en papel, lo que lleva a retrasos y confusiones en la administración de los libros.”

Dentro del mismo archivo, se nos indica que la biblioteca enfrenta diversos desafíos en la gestión de sus materiales de estudio, lo que afecta su capacidad para garantizar la disponibilidad y acceso oportuno de los mismos para los estudiantes y docentes. Algunos de los principales problemas que enfrenta la biblioteca permite identificar los siguientes **requerimientos del negocio**:

- **Falta de trazabilidad adecuada:** El sistema actual no permite llevar un registro detallado de los libros y otros materiales de estudio, lo que dificulta la ubicación y seguimiento de estos.
- **Dificultades para realizar búsquedas y reservas:** El sistema actual tiene problemas de indexación y búsqueda lo que obliga a los estudiantes y docentes a buscar los materiales de estudio de manera presencial.
- **Problemas en la administración de préstamos y devoluciones:** El sistema actual no cuenta con un registro completo y actualizado de los préstamos y devoluciones de los materiales de estudio, lo que dificulta la identificación de los libros vencidos o en mal estado, y la realización de seguimientos adecuados para garantizar la devolución de los materiales prestados.
- **Falta de seguridad y protección de datos:** El sistema actual no cuenta con medidas adecuadas de seguridad y protección de datos, lo que pone en riesgo la información personal de los usuarios y la integridad de los registros de la biblioteca.

Tomando en cuenta estos problemas, el archivo de contexto nos informa cuáles son algunos de los requisitos funcionales y no funcionales.

Comenzando con los **requisitos funcionales**, se tienen los siguientes 5:

1. **Búsqueda en línea de materiales de estudio:** El sistema debe contar con una funcionalidad de búsqueda en línea que permita a los usuarios buscar y reservar materiales de estudio desde cualquier dispositivo con conexión a internet.
2. **Reservas en línea de materiales de estudio:** El sistema debe permitir a los usuarios reservar materiales de estudio cuando no se encuentren disponibles al momento de la búsqueda.
3. **Registro detallado de materiales de estudio:** El sistema debe permitir llevar un registro detallado de los materiales de estudio manejando información como: ubicación y disponibilidad.
4. **Registro detallado de préstamos y devoluciones:** El sistema debe permitir llevar un registro detallado de los préstamos y devoluciones de los materiales de estudio, incluyendo información como fecha de préstamo, fecha de devolución, estado del material y usuario que realizó el préstamo.
5. **Alertas para préstamos vencidos o en mal estado:** El sistema debe enviar alertas automáticas para los materiales de estudio vencidos o en mal estado, para permitir una administración más eficiente y oportuna de los mismos

Se presentaron leves diferencias entre ambos grupos sobre cuáles eran prioritarios, por lo que se definió en conjunto la siguiente tabla de priorización:

Requerimiento funcional	Grupo 7	Grupo 8	Priorización final
Búsqueda en línea de materiales de estudio	Media	Alta	Alta
Reservas en línea de materiales de estudio	Alta	Media	Alta
Registro detallado de materiales de estudio	Alta	Alta	Alta
Registro detallado de préstamos y devoluciones	Baja	Baja	Baja

Alertas para préstamos vencidos o en mal estado	Baja	Baja	Baja
---	------	------	------

Mientras que con los **requisitos no funcionales**, se nos informan los siguientes 3:

1. **Control de acceso y seguridad:** El sistema debe contar con medidas de seguridad adecuadas para proteger los datos personales de los usuarios y los registros de la biblioteca, y debe incluir un control de acceso que permita a los usuarios acceder solo a la información relevante para ellos.
2. **Integración con sistemas externos:** El sistema debe permitir la integración con sistemas externos relevantes para la biblioteca, como sistemas de gestión académica y bases de datos de publicaciones científicas.
3. **Fácil mantenimiento y actualización:** El sistema debe ser fácil de mantener y actualizar en el futuro, para garantizar una gestión eficiente y sostenible de los materiales de estudio en la biblioteca.

En la priorización no existe diferencia entre los grupos, y esta se muestra en la siguiente tabla:

Requerimiento no funcional	Grupo 7	Grupo 8	Priorización final
Control de acceso y seguridad	Alta	Alta	Alta
Integración con sistemas externos	Baja	Baja	Baja
Fácil mantenimiento y actualización	Baja	Baja	Baja

Entre ambos grupos se tienen **10 casos de uso** (5 por cada equipo), varios son bastante similares y para esta entrega se eligieron los que responden a los requisitos de mayor prioridad.

Caso de uso 1

ID y nombre: UC-2 Buscar material de estudio

Autor: Amanda

Fecha: 17-4-2023

Actores involucrados: Estudiante/docente

Descripción: El estudiante ingresa información sobre el material de estudio (título, autor, etc) y el sistema le muestra una lista de artículos relacionados

Trigger: El estudiante ingresa información a buscar

Precondiciones:

- PRE1: Se ha autenticado correctamente al usuario en el sistema
- PRE2: Deben existir materiales de estudio en la base de datos del sistema

Post Condiciones:

- POST-1: El sistema busca entre la información disponible
- POST-2: Aparece una lista de materiales relacionados con la búsqueda

Flujo Normal:

1. El estudiante ingresa información clave a buscar
2. El sistema muestra información relevante a la búsqueda

Excepciones:

No se encuentra información similar entre el material en el sistema: Se debe mostrar que no se encontró ningún resultado

Prioridad y frecuencia de uso: Alta, 1000 veces por día

Caso de uso 2

ID y nombre: UC03 - CRUD del administrador

Autor: Javier Norambuena, Bastian Cornejo

Fecha: 17/04/2023

Actores involucrados: Administrador bibliotecario

Descripción: Los materiales de estudio pueden ser ingresados, revisados, actualizados o eliminados por el administrador bibliotecario

Trigger: El administrador bibliotecario ingresa a la página.

Precondiciones :

1. El administrador bibliotecario debe contar con los privilegios necesarios

Flujo normal:

1. El administrador bibliotecario está en la página principal.
2. Selecciona la opción de ingresar material de estudio.
3. Ingresa los datos del material de estudio.

Flujo alternativo 1:

1. El administrador bibliotecario está en la página principal
2. Busca el recurso de interés
3. Muestra recursos que coinciden con la búsqueda.
4. Se selecciona la opción revisar del recurso de interés

Flujo alternativo 2:

1. El administrador bibliotecario está en la página principal
2. Busca el recurso de interés
3. Selecciona la opción de modificar material de estudio

Flujo alternativo 3:

1. El administrador bibliotecario está en la página principal
2. Busca el recurso de interés
3. Selecciona la opción de eliminar material de estudio

Excepciones: Usuarios sin los permisos no pueden realizar CRUD.

Prioridad y frecuencia de uso:

1. Alta
2. Se utiliza las 24hrs los 7 días de la semana.

Postcondiciones: El recurso es modificado, creado o eliminado

Caso de uso 3

ID y nombre: UC-5 Actualizar registro de devolución

Autor: Benjamin

Fecha: 17-4-2023

Actores involucrados: Funcionario

Descripción: El funcionario actualiza los registros de una devolución dado que un estudiante devuelve el material de estudio prestado.

Trigger: El estudiante devuelve el material de estudio prestado.

Precondiciones:

1. PRE1: El funcionario está autorizado para actualizar registros de devolución.
2. PRE2: El material de estudio a devolver debe pertenecer a la biblioteca de la institución.

Post Condiciones:

1. POST-1: El sistema registra la devolución del material de estudio.
2. POST-2: El sistema actualiza el estado y disponibilidad del material de estudio devuelto indicando la fecha de devolución.

Flujo Normal:

1. El estudiante hace entrega del material de estudio prestado.
2. El funcionario actualiza el registro de devolución del material de estudio.
3. El sistema indica que la actualización fue exitosa.

Excepciones: El sistema no reconoce que el material de estudio devuelto fue solicitado en el pasado.

Prioridad y frecuencia de uso: Alta, 600 veces por día.

Implementación y desarrollo

Los casos de uso elegidos reflejan casi la totalidad del flujo de trabajo que tanto el bibliotecario/administrador y el estudiante realizan. Los puntos pendientes, para que el flujo quede completo, quedan para el siguiente ciclo de desarrollo.

Para ofrecer una **solución** a este problema, se decide desarrollar una nueva **aplicación para la gestión de la biblioteca**. La aplicación está **compuesta** por una **API** encargada de realizar todo el manejo y manipulación de la base de datos, para acceder a la API se construye una **interfaz web** en la cual los usuarios pueden realizar sus flujos de trabajo.

La API se desarrolló utilizando Flask para construir los métodos y una base de datos SQLite. Para el front end, se desarrolló utilizando React junto a los paquetes Bootstrap y Wouter. Bootstrap fue utilizado para generar una interfaz más amigable al usuario y que se pueda ajustar a diferentes tipos de dispositivos, por otro lado Wouter fue utilizado para generar las rutas dentro de la interfaz.

Uno de los requisitos no funcionales de alta prioridad es el tema de la **seguridad** y el manejo de los datos guardados en la base de datos (quién puede editar la base de datos y cómo), para ello se desarrolló un **sistema** que utiliza **tokens de acceso** para utilizar los métodos de la API, ver figura 1. La idea de esto es simple, el usuario al loguearse envía sus credenciales a la API y las valida, cuando esto ocurre al usuario se le envía un token que es almacenado de forma local en el navegador, en cada llamada este token es adjuntado junto a los otros datos. En caso de no tener tokens válidos, los métodos llamados no se ejecutan.

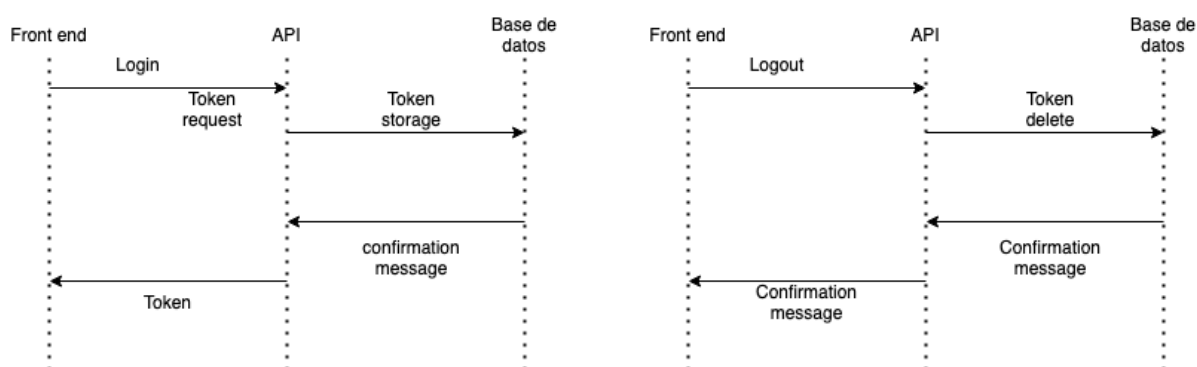


Figura 1: Esquema de login/logout de la aplicación

Respecto al front end, como se mencionó anteriormente, se desarrolló utilizando React, un framework desarrollado en Javascript. La razón de esta elección fue estar en concordancia con **Jest**, el **framework de pruebas**. La **dependencia entre una aplicación React y Jest** se establece a través de la instalación de Jest como una dependencia de desarrollo en el proyecto, la configuración de Jest para ejecutar pruebas de la aplicación React y la ejecución de las pruebas utilizando Jest. **No hay dependencia con el backend** ya que en ningún momento la herramienta le habla a la API, todo se hace con el frontend como intermediario.

Pruebas del software

Estrategia de pruebas utilizadas

Antes de hablar de las estrategias, es menester indicar cuáles fueron los supuestos que se tomaron en cuenta al momento de desarrollar las pruebas.

1. Al momento de realizar las pruebas los usuarios están logueados, es decir, dentro de las pruebas no se considera el login como una parte del flujo.
2. El backend es simulado dentro de la interfaz, debido a que este está construido en Python, esto se traduce a que se asume que las respuestas de la API siempre serán correctas.

La estrategia principal es concentrarse en el frontend y sus componentes, es decir, probar que el frontend se renderiza de forma correcta y que los componentes que se comunican con la API funcionan y retornan lo que deben retornar.

Procedimiento de ejecución de pruebas

Considerando la estrategia antes mencionada, se decidió crear 4 suites de casos que se encarguen de verificar el correcto funcionamiento de ciertos componentes.

1. Renderizado del inicio
2. Agregar material
3. Editar material
4. Eliminar material

Entre las 4 suites se tienen 7 pruebas unitarias en total.

Reporte de resultados

Los resultados que se muestran a continuación fueron los obtenidos de las pruebas realizadas el día Jueves 18 de Mayo del año 2023, esto después de corregir varios errores que fueron detectados en la presentación del día Viernes 12 de mayo del año 2023.

Test	Descripción	Esperado	Obtenido	Comentarios
TC_01	Render de página de inicio	Mensaje de bienvenido	Mensaje de bienvenido	Prueba exitosa, sin comentarios
TC_02	Muestra todos los libros	Se muestran todos los libros disponibles	Se muestran todos los libros	Prueba exitosa, sin comentarios
TC_03	Busca libro en particular	Muestra libro en específico	Muestra libro en específico	Prueba exitosa, sin comentarios
TC_04	Muestra detalle de un libro en particular	Se muestra detalle de libro	Se muestran detalles del libro	Prueba exitosa, sin comentarios
TC_05	Agrega un libro al sistema	Se crea un nuevo libro en el sistema	Se crea un nuevo libro	Prueba exitosa, error en manejo de procesos
TC_06	Edita un libro existente	Datos de libro modificados	Libro modificado	Prueba no exitosa
TC_07	Elimina un libro existente	Libro eliminado del sistema	Libro eliminado	Prueba exitosa, sin comentarios

En la tabla, se puede observar el resultado de las pruebas realizadas utilizando Jest, de las 7 pruebas realizadas, 6 tuvieron un resultado positivo mientras que la prueba TC_06 tuvo un resultado fallido.

Detalles de la organización y el trabajo

Estructura del equipo

Para el desarrollo del trabajo, hubo una división de los roles, donde Javier y Benjamin tomaron el rol de Fullstack y se concentraron en el desarrollo del front y back, por otro lado, Bastián se encargó en el aprendizaje y desarrollo de las pruebas utilizando Jest y herramientas de testing de React.

Para el almacenamiento del código, se utilizó un repositorio dentro de una organización en GitHub (<https://github.com/validacion-y-verificacion/presentacion>), para la comunicación del equipo (texto y voz) se utilizó en chat en Discord, no se definió una periodicidad para las reuniones, después de la reunión se decidía cuando sería la siguiente según las tareas que se tenían que llevar a cabo y la deadline. Finalmente, para la estructura y designación de las tareas se utilizó, en las últimas instancias, Jira, mientras que durante las primeras etapas sólo se asignaban mediante canales de texto.

A continuación se muestra una tabla con los tiempos dedicados por cada integrante en las diferentes tareas.

Actividad/Persona	Javier	Bastián	Benjamin
Desarrollo de la aplicación	24	0	8
Desarrollo de pruebas	0	23	0
Reuniones	4	4	4
Total	28	27	12

En los desarrollos se incluye el estudio, el diseño, la programación y la corrección de errores.

Problemas y soluciones encontrados

Uno de los problemas más serios que se tuvo a la hora de desarrollar el proyecto fue el tema de los tiempos, dado que debido a esto el desarrollo de la aplicación fue muy tardío dando poco tiempo para desarrollarla en extenso. Frente a esto, la solución encontrada fue dejar de lado elementos deseables como el deployment en la nube y trabajar solamente en local, también, se acotó el proyecto para concentrarnos en casos de uso que reflejen de la mejor manera los requerimientos indicados con alta prioridad. De esta manera, el proyecto fue acotado al máximo para poder cumplir con los tiempos y dejar un margen para el desarrollo y construcción de las pruebas usando Jest.

Para el desarrollo se decidió utilizar código fuente de un proyecto anterior que utilizaba React y a la hora de traspasarlos a este nuevo contexto usar el ChatGPT en su versión gratuita para adaptarlo.

Finalmente, el último problema es el desarrollo de la API en un framework de Python, lo cual genera un problema ya que no podemos probar las rutas y el funcionamiento de la API con Jest de forma apropiada, dado que sólo podríamos comprobar los llamados realizados al backend y las respuestas recibidas por este mientras este se ejecuta por detrás, lo cual no sería lo que se esperaría a la hora de probar del backend cuando herramientas como pytest permitirían probar de mejor manera el backend sin la necesidad de ejecutar este durante las pruebas, por lo que se decidió simular las respuestas de la API que solicitaba el front y sólo probar este último.

Evidencia

Github

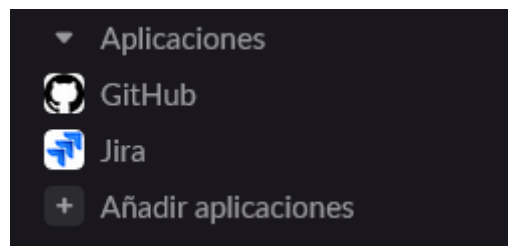
En el siguiente repositorio se encuentra el código del proyecto, <https://github.com/validacion-y-verificacion/presentacion>, se utilizó una diferenciación de 2 ramas y todo el resto son generadas a partir de estas dos, que son main y develop. Cuando se agregan nuevos features, se generaba una nueva rama a partir de develop y cuando se hacía el push, dentro del repositorio se llevaba a cabo el pull request a develop.



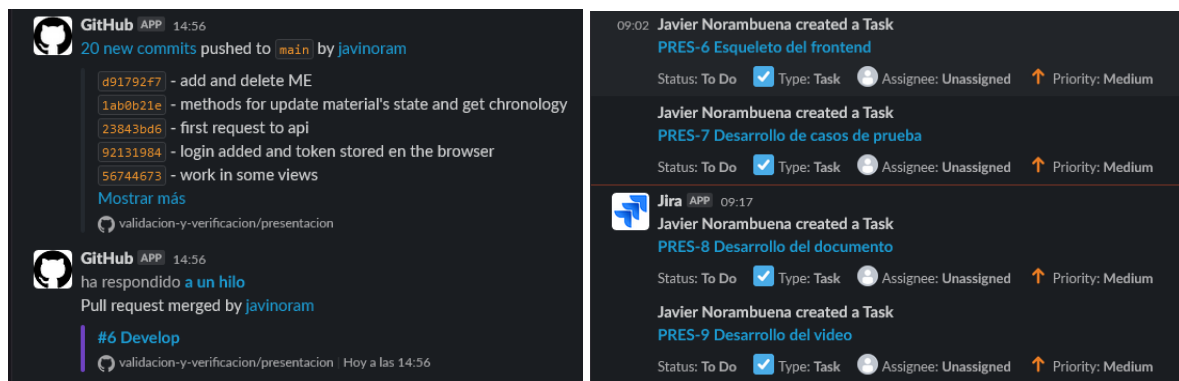
En la imagen de arriba se puede observar el comportamiento de las ramas y cómo se iban uniendo durante el proyecto.

Slack/Discord

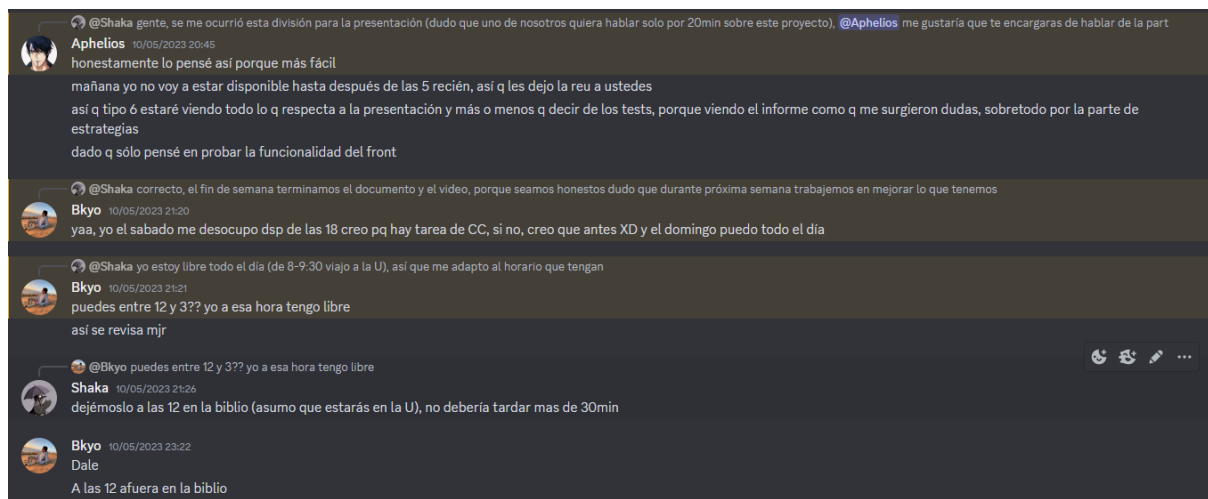
Si bien no se utilizó el slack, si se logró la integración con github y con jira.



En la imagen de arriba se puede ver que las aplicaciones están instaladas dentro del canal de Slack. Por otro lado, en las imágenes de abajo se puede observar cómo los bots informan sobre los cambios dentro de Jira y GitHub, mostrando así que están correctamente integrados.

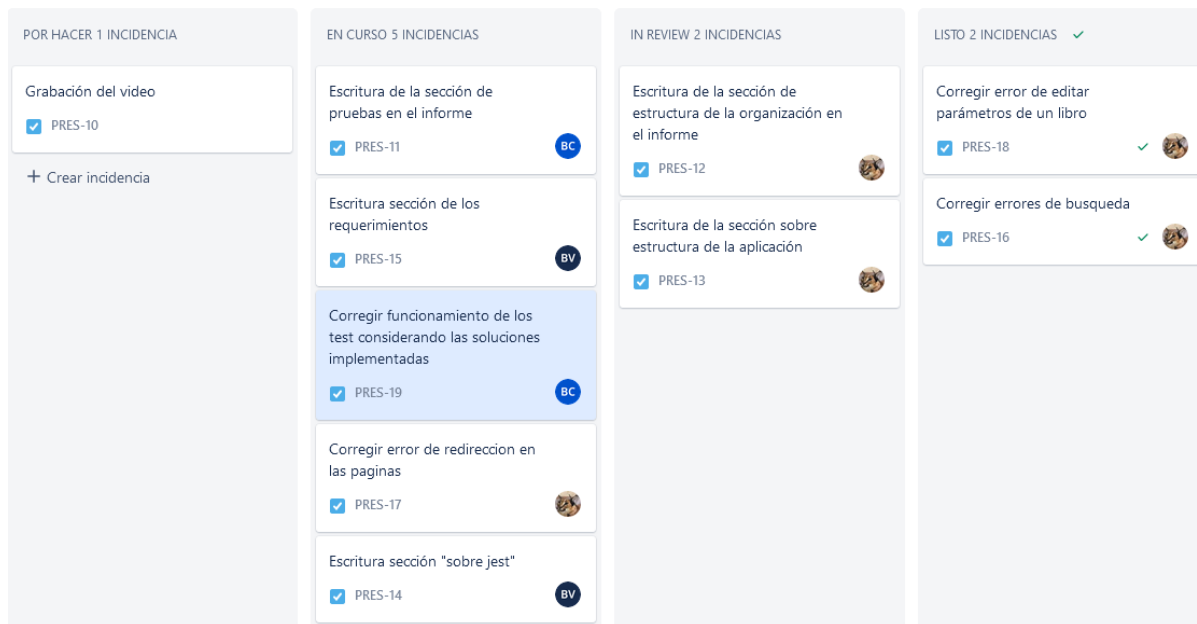


Finalmente, las imágenes a continuación son respecto al chat de Discord que utilizamos para coordinarnos, tener reuniones y organizar el proyecto.



Jira

Finalmente tenemos el tablero de Jira, el cual fue utilizado para la parte final del proyecto, es decir, arreglar errores dentro de los flujos de los casos de uso y corrección de errores menores en los test, organizar el informe final y el video explicativo. La imagen de abajo es el tablero que se tiene a la fecha (13/05/2023), donde cada integrante tiene asignada las tareas que debe realizar.



POR HACER 1 INCIDENCIA	EN CURSO 5 INCIDENCIAS	IN REVIEW 2 INCIDENCIAS	LISTO 2 INCIDENCIAS ✓
Grabación del video ☒ PRES-10 + Crear incidencia	Escritura de la sección de pruebas en el informe ☒ PRES-11 BC Escritura sección de los requerimientos ☒ PRES-15 BV Corregir funcionamiento de los test considerando las soluciones implementadas ☒ PRES-19 BC Corregir error de redirección en las paginas ☒ PRES-17 Escritura sección "sobre jest" ☒ PRES-14 BV	Escritura de la sección de estructura de la organización en el informe ☒ PRES-12 Escritura de la sección sobre estructura de la aplicación ☒ PRES-13 	Corregir error de editar parámetros de un libro ☒ PRES-18 ✓ Corregir errores de búsqueda ☒ PRES-16 ✓